Expressivity Engineering for Single-Layer Reinforcement Learning

A Controlled TriGLU Study on Qwen3-1.7B-Base

Hanting Wu
Independent Research

July 2026

Abstract

We introduce *Expressivity Engineering* (EE) from a multiplication-first distinction: a static linear map performs pure channel mixing, whereas multiplying a representation by itself, or by a transformed version of itself, creates higher-order and input-conditioned interaction. Under this lens, nonlinear activation functions, GLU-family gates, attention, routed MoE/MoLoRA, and HyperNetworks are not disconnected tricks; they are different realizations of input-derived multiplication or dynamic weighting. To test this proposal under a bounded compute budget, we use the layer-selective reinforcement learning workflow of Zhang et al. [2026] as experimental guide rails: Qwen3-1.7B-Base is trained with Group Relative Policy Optimization while only decoder Layer 10 is updated. The EE intervention retains the original SwiGLU path and adds an exact-no-op-initialized, FP32 side network that multiplicatively modulates the Layer-10 SwiGLU hidden state. The implementation, called TriGLU in code and more descriptively ToTGLU in this report, adds 20,986,368 parameters, or 41.69% of one Qwen3-1.7B decoder layer.

We compare the intervention with a matched naive whole-Layer-10 baseline. Both runs start from the same untuned Qwen3-1.7B-Base revision and share data order, initialization and rollout seeds, reward, response cap, and a six-GPU `504/126/6` train/mini/micro-batch contract. Corrected evaluations are reported at matched global steps 158, 196, 226, 256, and 294. Averaged across these correlated checkpoints, TriGLU improves MathAvg from 52.224 to 52.997 (+0.773 points) and the provisional CodeAvg from 32.671 to 33.659 (+0.988), while Reasoning and Language category averages are approximately tied (-0.213 and -0.128). The clearest gains occur on harder generative tasks: **OlympiadBench** (+2.548), **corrected LiveCodeBench** (+1.063), and a manually audited **GPQA-Diamond-Freeform** diagnostic (+1.746). These results are promising but not definitive: they come from one training seed, checkpoint variation is not independent-seed uncertainty, several evaluation protocols required correction, and response-length caps remain in both training and evaluation. The study therefore establishes an artifact-backed positive pilot for EE-PEFT, not a general scaling-law claim.

1 Introduction

Expressivity Engineering begins from one operation: multiplication. For a representation x, a static linear map Wx performs channel mixing. It can recombine coordinates, but a single such map does not create products between input-dependent features. Multiplying a representation by itself, or by a learned preprocessing of itself, changes that algebraic structure. Expressions such as $f(x) \odot g(x)$ contain feature products; expressions such as $W(x)x$ use the current state to construct the weights that act on that same state. We take this self- or input-conditioned multiplication, rather than a list of particular modules, as the primitive of *Expressivity Engineering*.

This primitive reveals a shared structure beneath mechanisms that are usually placed in separate literatures. A nonlinear activation can be approximated over a bounded operating range by polynomial terms, whose powers are repeated self-multiplications. GLU and SwiGLU multiply learned feature streams [Shazeer, 2020]. Attention constructs an input-dependent weight matrix from queries and keys and multiplies it into values [Vaswani et al., 2017]. HyperNetworks generate the parameters that act on another representation [Ha et al., 2016]. MoE and MoLoRA multiply expert outputs by routed, input-dependent coefficients [Shazeer et al., 2017, Zadouri et al., 2024]. These are not mathematically identical modules, but they instantiate the same engineering move: replace only fixed linear channel mixing with computation whose effective transform is conditioned on, and multiplied with, the current representation.

We propose *Expressivity Engineering* as a unifying engineering lens for this move. It is a working taxonomy rather than a theorem that totally orders neural architectures by expressive power. It is useful because it separates two

questions:

1. How much additional input-dependent interaction is introduced?
2. Where can that interaction be inserted and trained economically?

The present study addresses the second question through layer-selective RL. Zhang et al. [2026] show that RL gains in Qwen3 and related models are concentrated in a small set of middle layers; on Qwen3-1.7B-Base, Layer 10 is their strongest single-layer math result. That finding is not the conceptual origin of EE. It supplies the empirical localization result and workflow guide rails that make a compute-bounded test of our independently proposed EE hypothesis feasible: instead of pretraining a new architecture from scratch, we can add expressivity at one high-contribution layer and compare it against a naive update of exactly the same layer.

Our contributions are:

- a multiplication-first EE taxonomy connecting general nonlinear activations, GLU-family FFNs, attention, HyperNetworks, routed full and low-rank experts, grid-wise modulation, and explicit polynomial modules;
- an exact-function-preserving Layer-10 TriGLU/ToTGLU intervention for Qwen3-1.7B-Base;
- a matched six-GPU GRPO comparison from the same untuned base model, with deterministic data exposure and five common evaluation checkpoints;
- corrected Math and out-of-distribution evaluation records, including a strict free-form GPQA-Diamond audit and explicit protocol/cap-hit boundaries;
- a systems account of registered Hugging Face/vLLM integration, continuous batching, live weight synchronization, and the current serving-performance cost of the FP32 side branch.

2 Expressivity Engineering

2.1 A working definition

For an input vector x, the baseline operation is fixed linear channel mixing, $y = Wx$. The defining EE move is instead to multiply an input-derived quantity back into the same representation or into another transform of it:

$$y = f(x) \odot g(x) \quad \text{or} \quad y = W(x)x. \tag{1}$$

The first form includes literal self-interaction when $f = g$, and interaction with a preprocessed self when f and g are different transforms of the same input. The second is the matrix-valued version: x generates some or all of the weights that are then applied to x. Familiar mechanisms can be written as instances of these forms:

$$\text{activation:} \quad \phi(z) \approx \sum_{k=0}^{K} a_k z^{\odot k}, \tag{2}$$

$$\text{GLU:} \quad y = \phi(Ax) \odot Bx, \tag{3}$$

$$\text{attention:} \quad Y = \operatorname{softmax}\left(\frac{Q(X)K(X)^\top}{\sqrt{d}}\right) V(X), \tag{4}$$

$$\text{dynamic weights:} \quad y = (W + \Delta W(x))\, x, \tag{5}$$

$$\text{routed experts:} \quad y = \sum_{e} \pi_e(x) f_e(x). \tag{6}$$

Equation 2 is an approximation statement on a bounded domain, not a claim that every activation is globally polynomial. It makes the key algebraic point: nonlinear activation functions introduce powers and self-interactions that a lone linear channel mixer cannot. GELU, for example, is explicitly $z\Phi(z)$ [Hendrycks and Gimpel, 2016]; PolyCom/PolyNorm makes the polynomial view trainable and explicit [Zhuo et al., 2024]. Under this definition:

- general nonlinear activations are the most local form of EE: they turn each channel into a nonlinear self-interaction before later channel mixing; explicit polynomial activations expose this structure rather than creating it from nothing;
- GLU/SwiGLU makes the multiplication visible by gating one learned stream with another [Shazeer, 2020];
- attention is dynamic weighting at token level: $Q(X)$ and $K(X)$ construct weights from the current sequence, and those weights multiply $V(X)$ [Vaswani et al., 2017];
- HyperNetworks generate parameters for another network [Ha et al., 2016];

- HyperGrid decomposes task-conditioned modulation into grid-wise projections [Tay et al., 2020];
- HyperFormer generates task/layer/position-conditioned adapter parameters, and HyperLoRA generates constrained low-rank adapters [Mahabadi et al., 2021, Lv et al., 2024];
- MoE multiplies full expert outputs by router coefficients [Shazeer et al., 2017], while MoLoRA applies the same routed-mixture pattern to low-rank updates [Zadouri et al., 2024]. At the routing level, a conventional linear-logit MoE router is the full-expert analogue of linearly routed MoLoRA; their expert parameterizations, capacities, and compute profiles remain different;
- PolyCom/PolyNorm makes higher-order activation interactions explicit and trainable [Zhuo et al., 2024].

The claim is common primitive, not module equivalence. HyperFormer and HyperLoRA are primarily task-conditioned parameter generators; attention is a per-token dynamic weight matrix over values; MoE routes among full experts; MoLoRA routes among low-rank updates; HyperGrid modulates structured regions of weight matrices; and PolyNorm uses elementwise polynomial powers and normalization. The present TriGLU branch is another point in this family: it computes a per-token multiplicative modulation of one existing FFN hidden state.

2.2 Scope beyond Transformers

The EE lens is not limited to language models. Any architecture containing an MLP can, in principle, be augmented with multiplicative, routed, polynomial, or dynamically parameterized computation. For Transformer FFNs, such operations remain linear in sequence length, unlike dense self-attention's quadratic token-pair interaction. This does not make EE free: added FFN branches increase FLOPs, activation memory, kernel-launch overhead, and optimization difficulty. It does mean that richer channel computation can be explored without directly increasing the number of attention score pairs or the KV-cache length.

Attention is therefore not merely adjacent background; it is a canonical EE mechanism at token-mixing scale. Its effective matrix $\mathrm{softmax}(QK^\top/\sqrt{d})$ is generated from the current hidden states before it weights the values. This controlled study nevertheless keeps the attention architecture unchanged: only the Layer-10 FFN computation differs between conditions, while the ordinary Layer-10 attention parameters remain trainable in both. The experiment tests whether the same broad multiplication-first principle can be pushed further inside the FFN without changing attention.

3 Model

3.1 Backbone and update scope

The backbone is Qwen3-1.7B-Base [Yang et al., 2025]: 28 decoder layers, hidden width 2,048, FFN intermediate width 6,144, 16 query heads, 8 key/value heads, and a 32,768-token context window. Both the baseline and EE variant update decoder Layer 10 only. Layer-10 attention projections, RMSNorm parameters, and all original FFN projections are trainable. Embeddings, the language-model head, and all other decoder layers are frozen.

This is a stricter comparison than using the untuned base model as the primary control. The question is not whether RL helps, but whether adding a particular expressivity mechanism improves a strong naive single-layer RL update.

3.2 TriGLU/ToTGLU side modulation

Let $x \in \mathbb{R}^{2048}$ be the Layer-10 FFN input. The original SwiGLU hidden state is

$$h_0 = \mathrm{SiLU}(W_g x) \odot (W_u x), \qquad h_0 \in \mathbb{R}^{6144}. \tag{7}$$

The FP32 side network first compresses to a 512-dimensional bottleneck, $z = Ax \in \mathbb{R}^{512}$, and forms three 2,048-dimensional streams:

$$v = Vz, \tag{8}$$
$$g = \mathrm{SiLU}(Gz), \tag{9}$$
$$q = \mathrm{SiLU}(F_2\,\mathrm{GeLU}(F_1 z))\,. \tag{10}$$

Their product is projected to the backbone intermediate width,

$$s = U(v \odot g \odot q) \in \mathbb{R}^{6144}, \tag{11}$$

and modulates the original hidden state:

$$h = h_0 \odot [1 + \alpha \tanh(s)], \qquad y = W_d h, \qquad \alpha = 0.1. \tag{12}$$

The code calls this module TriGLU. Because the base SwiGLU already contains two multiplicative streams and the side branch multiplies three more, we use ToTGLU ("TriGLU on TriGLU") when emphasizing its five-stream interpretation. The name is descriptive rather than a claim that the module implements a standard architecture from prior work.

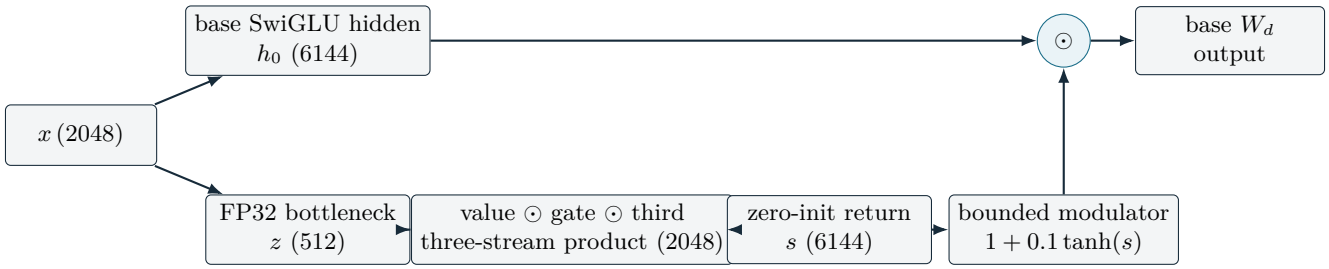

Figure 1: Layer-10 TriGLU/ToTGLU intervention. The original SwiGLU path is retained. A per-token FP32 side network produces a bounded multiplicative modulator before the original down projection.

3.3 Exact no-op initialization and parameter cost

The side return projection U (named `triglu_side.up` in code) is initialized to zero. Therefore $s = 0$, the multiplier in Eq. 12 is exactly one, and the initial model is functionally identical to the unmodified backbone. This invariant isolates subsequent behavior from a random architecture shock.

The active dimensions are `side_dim=512` and `side_hidden=2048`. The side network contains 20,986,368 trainable parameters, 41.69% of the 50,337,792-parameter selected decoder layer. The base Layer-10 parameters remain jointly trainable. This is parameter-efficient relative to full-model RL, but it is deliberately a high-capacity intervention rather than a minimal adapter.

4 Experimental Design

4.1 Relationship to the layer-selective RL workflow

The experiment follows the operational structure of Zhang et al. [2026]: Qwen3-1.7B-Base, Layer 10, NuminaMath-CoT, GRPO, single-layer freezing, a 3,072-token response cap, and Math plus OOD benchmark families. The original work reports that Layer 10 reaches 51.8 MathAvg versus 50.8 for full-parameter RL under its protocol. Its results and hyperparameters are guide rails, not copied output: its executable repository and exact evaluation harness were not available to this study, so absolute score parity is not claimed.

Our scientific variable is the Layer-10 FFN architecture. The baseline updates the ordinary Layer-10 Transformer block; the EE condition updates that same block plus the side network. Both begin from the same untuned base model.

4.2 Data and decontamination

Training uses a 50,000-problem materialization derived from NuminaMath-CoT [Li et al., 2024]. A decontamination pass removes question-level overlap with the evaluation sets before training. The two variants use the same parquet hashes, deterministic shuffled permutation, omitted-row set, initialization seed, and rollout seed (`20260707`). This controls data exposure within the comparison; it does not prove that the local data processing exactly matches an unavailable external implementation.

Table 1: Six-rank production contract.

Field	Value
Train batch	504 prompts
PPO mini batch	126 prompts
PPO micro batch	6 prompts
Group size	4 responses/prompt
Responses per global update	2,016
Prompts through step 98	49,392
Deterministically omitted at step 98	608 / 50,000
Rollout replicas	6 independent TP=1 replicas

4.3 GRPO and the six-GPU batch contract

GRPO samples a group of responses per prompt and normalizes their advantages within the group, avoiding a learned critic [Shao et al., 2024]. The implementation uses veRL/HybridFlow [Sheng et al., 2025] with vLLM rollouts [Kwon et al., 2023]. The paper-aligned tuple is train/mini/micro batch `512/128/8`, group size 4, KL coefficient 0.001, clip range 0.2, and four epochs. The actual six-RTX-5090 matched run uses the explicitly approved divisible tuple in Table 1.

The loader uses `drop_last=True`; step 98 is therefore a *near-one-pass* milestone, not an exact epoch. No irregular tail batch is introduced. Both architectures omit exactly the same 608 rows, giving a matched comparison with a 1.56% exposure deviation from 512×98.

4.4 Training stages and post-hoc interventions

The trajectory contains three 98-update stages. The first uses the original 5×10^{-6} learning rate. The second keeps that rate through step 128 and then applies cosine decay to 5×10^{-7} at step 196. The third decays from 5×10^{-7} to 5×10^{-8} at step 294. Each variant's frozen KL reference is reset to its own step-98 policy for updates 99–196 and to its own step-196 policy for updates 197–294.

The decay and reference resets were introduced during the live research cycle, not preregistered before update 1. Step 158 is the first reported common checkpoint after the step-128 schedule change and the first point where the architectural trajectories visibly separated. We report all available matched major checkpoints from 158 onward rather than selecting a single best model.

4.5 Historical SFT pilot

Before the main RL wave, a two-epoch, 50K-example supervised fine-tuning pilot compared SHS, the whole-layer baseline, TriGLU, and OFT. It completed operationally, but did not provide a reliable architecture selection signal. SHS, baseline, and TriGLU had four-task Math averages within 0.32 points. OFT achieved the lowest teacher-forced validation loss (0.5893) yet collapsed under autoregressive evaluation (13.04 Math average versus approximately 43 for the other three). This is direct evidence in this project that same-distribution SFT loss is not a sufficient proxy for the intended generative reasoning behavior. Consequently, the main GRPO comparison starts from the untuned base, not from an SFT checkpoint.

OFT [Qiu et al., 2023] was scaffolded but not run as a GRPO comparator. The implemented policy freezes the original Layer-10 SwiGLU matrices and trains orthogonal rotations, whereas TriGLU jointly trains the original SwiGLU and the side branch. A direct comparison would therefore confound architectural expressivity with different backbone adaptation constraints. OFT remains a useful future control under a matched update policy.

5 Evaluation

5.1 Benchmark suite and aggregate definitions

The in-domain Math category follows MATH-500 [Hendrycks et al., 2021], GSM8K [Cobbe et al., 2021], OlympiadBench [He et al., 2024], and the 2023 AMC 10/12 competition set [Mathematical Association of America, 2023]. MathAvg is the unweighted mean of those four scores and uses AMC Average@32, not AMC greedy pass@1. AMC greedy is retained as a modal-path diagnostic.

OOD evaluation includes:

- Code: corrected HumanEval+ [Liu et al., 2023], provisional MBPP [Austin et al., 2021], and corrected LiveCodeBench [Jain et al., 2024];
- Reasoning: GPQA-Diamond [Rein et al., 2023] and MMLU-Pro [Wang et al., 2024], plus a separate GPQA-Diamond-Freeform manual diagnostic that is not included in ReasoningAvg;
- Language and instruction following: C-Eval [Huang et al., 2023], IFEval [Zhou et al., 2023], and MGSM [Shi et al., 2022].

Except AMC Average@32, the primary corrected benchmark cells use greedy decoding. This choice is especially important for the FP32 TriGLU model, whose sampled generations at temperature 1 were less stable. The report does not convert that observation into a deployment recommendation; temperature and logit-distillation behavior require a dedicated study.

5.2 Benchmark semantics and relative difficulty

The suite intentionally spans different capability regimes. "Difficulty" below is a qualitative task-level description, not a claim that scores are directly comparable across datasets.

Table 2: Benchmark roles in the evaluation suite.

Benchmark	Task and answer format	Relative regime
MATH-500	Competition-style secondary mathematics; free-form numeric or symbolic answer	Harder than routine school arithmetic; broad algebra, geometry, number theory, and combinatorics
GSM8K	Grade-school arithmetic word problems; free-form short answer	Elementary mathematics with multi-step linguistic reasoning
OlympiadBench	Olympiad-oriented mathematics; free-form derivation and answer	Hardest in-domain mathematical set in this report
AMC 2023	AMC 10/12 high-school contest questions; multiple choice	Between routine K–12 work and olympiad-style proof problems
HumanEval+	Function synthesis from a specification; execution-based tests	Compact code generation with strengthened hidden tests
MBPP	Short natural-language programming tasks; execution-based tests	Introductory-to-intermediate code generation; score remains provisional here
LiveCodeBench	Time-indexed competitive-programming problems; execution-based tests	Harder and less contamination-prone coding evaluation
GPQA-Diamond	Graduate-level science questions; four-way multiple choice	Expert factual and scientific reasoning; 25% random-choice floor
MMLU-Pro	Ten-choice, multi-domain knowledge and reasoning	Broad professional/academic reasoning with a 10% random-choice floor
GPQA-Diamond-Freeform	Same science questions without answer options; manual semantic audit	Stricter diagnostic that removes the multiple-choice floor
C-Eval	Chinese multi-subject multiple choice	Broad school-to-professional knowledge in Chinese
IFEval	Verifiable instruction-following constraints	Tests compliance rather than subject-matter depth
MGSM	Multilingual grade-school mathematics; free-form short answer	Elementary mathematics under cross-lingual generation pressure

5.3 Protocol corrections

Several legacy evaluation cells measured harness failures rather than model capability. The corrected table therefore uses:

- a raw no-chat HumanEval+ route after the chat-wrapped protocol caused Hebrew/template collapse in both TriGLU and the untuned base;
- a corrected LiveCodeBench sandbox output contract after the legacy contract produced an artificial all-zero result;
- explicit separation of AMC Average@32 from greedy pass@1;
- a row-by-row semantic audit of 1,260 GPQA-Diamond-Freeform responses, with uncertain judgments counted as incorrect.

These corrections preserve prompts and scoring intent, but do not establish paper-exact evaluation parity. MBPP still lacks a full protocol-matrix rerun; its score and the derived CodeAvg remain provisional.

5.4 Response-cap boundary

All Math benchmarks contain 3,072-token cap hits: 5.02% on MATH-500, 0.43% on GSM8K, 13.85% on OlympiadBench, 3.84% on AMC Average@32, and 8.25% on AMC greedy. Training rollouts also contain nonzero cap hits. Many inspected cases are repetition loops, but a minority could conceivably reach a correct answer if continued. We retain the paper-style cap-hit-inclusive results for matched comparison and do not describe them as uncapped capability ceilings.

OOD cap rates are more heterogeneous. HumanEval+ is 2.50%, LiveCodeBench 9.73%, GPQA-Diamond 9.85%, MMLU-Pro 4.73%, MGSM 8.61%, and GPQA-Diamond-Freeform 7.62%. MBPP (93.92%), C-Eval (68.31%), and

IFEval (39.09%) are severe exceptions that require protocol-aware reruns before strong absolute interpretation.

6 Results

6.1 Across-checkpoint summary

Table 3 reports the population mean and standard deviation across five matched late-stage checkpoints, not across independent training seeds. An independent multiple-seed sweep was outside the available compute budget: the study had six RTX 5090 GPUs and was scoped to days rather than weeks. The checkpoints were therefore drawn from the regime in which both training reward and downstream evaluation had plateaued and then oscillated rather than exhibiting a sustained trend. They sample local within-run variability from one seeded realization of the stochastic training process, including shuffled data order and rollout sampling. Because they belong to one optimization trajectory, the observations are serially dependent and may be positively correlated. This dependence does not invalidate their mean as a descriptive summary that reduces sensitivity to choosing one favorable checkpoint, but it lowers the effective sample size: the reported standard deviation measures checkpoint variation, not independent-seed uncertainty, a standard error, or a confidence interval.

This summary is operationally relevant because production model selection normally chooses the better validated checkpoint from a late-stage candidate pool rather than blindly deploying the final update. The across-checkpoint mean characterizes the average quality available within that selection window, and the per-checkpoint record shows the candidates from which a deployment choice can be made. It does not imply that every checkpoint is equally strong: the magnitude and even direction of the TriGLU–baseline difference vary by checkpoint, and the benchmarks showing the most prominent gains are not identical at every checkpoint.

Table 3: Corrected matched-checkpoint results (steps 158/196/226/256/294). Values are mean ± population standard deviation in percentage points. *MBPP and therefore CodeAvg remain provisional.
†All category averages are equal-weight descriptive summaries and are intentionally muted; the three bold rows are the hard generative benchmarks discussed in the text.
"Baseline" denotes the best layer from the source single-layer reinforcement-learning (SLRL) study—Layer 10—not the untuned base model and not full-parameter RL. A negative difference therefore means only that TriGLU trails this strongest SLRL comparator on that metric; it does not establish fundamental capability degradation below the untuned base. The source protocol reports Layer-10 SLRL at 51.8 MathAvg versus 50.8 for full-parameter RL, so parity or a small deficit relative to this baseline may still correspond to parity or an improvement relative to full RL, subject to the cross-harness boundary in Sections 3.1 and 4.1.

Metric	TriGLU	Baseline	Mean difference
MATH-500	67.520 ± 1.230	66.880 ± 0.412	+0.640
GSM8K	82.942 ± 0.322	82.396 ± 0.446	+0.546
OlympiadBench	$\mathbf{28.059 \pm 1.294}$	$\mathbf{25.511 \pm 0.533}$	**+2.548**
AMC 2023 Average@32	33.469 ± 0.755	34.109 ± 0.950	−0.640
MathAvg†	52.997 ± 0.184	52.224 ± 0.393	+0.773
AMC 2023 greedy	38.000 ± 1.000	35.500 ± 2.915	+2.500
HumanEval+ corrected	60.854 ± 0.896	59.512 ± 1.131	+1.342
MBPP*	29.280 ± 1.121	28.720 ± 0.844	+0.560
LiveCodeBench corrected	$\mathbf{10.845 \pm 0.401}$	$\mathbf{9.782 \pm 0.139}$	**+1.063**
CodeAvg*†	33.659 ± 0.260	32.671 ± 0.567	+0.988
GPQA-Diamond (MCQ)	26.566 ± 1.880	26.565 ± 2.273	+0.001
MMLU-Pro	33.895 ± 0.576	34.320 ± 0.198	−0.425
ReasoningAvg†	30.230 ± 0.691	30.443 ± 1.121	−0.213
GPQA-Diamond-Freeform	$\mathbf{6.349 \pm 1.328}$	$\mathbf{4.603 \pm 1.053}$	**+1.746**
C-Eval	47.548 ± 0.566	46.182 ± 0.560	+1.366
IFEval	27.421 ± 0.669	26.752 ± 0.657	+0.669
MGSM	56.538 ± 5.340	58.960 ± 0.199	−2.422
LanguageAvg†	43.836 ± 1.724	43.964 ± 0.334	−0.128

6.2 What the result supports

The primary Math result is positive: TriGLU improves MathAvg by 0.773 points across the five matched checkpoints. The gain is not uniform. It is largest on **OlympiadBench** and is accompanied by positive MATH-500 and GSM8K differences, while AMC Average@32 is lower. The separate greedy AMC diagnostic favors TriGLU by 2.5 points, illustrating that sampled distribution-wide reliability and the greedy modal path need not move together.

The OOD profile is also structured rather than uniformly positive. Corrected HumanEval+ and **LiveCodeBench** favor TriGLU, with the largest relative coding gain on the harder **LiveCodeBench** task. GPQA-Diamond MCQ is effectively tied, MMLU-Pro slightly favors the baseline, and strict **GPQA-Diamond-Freeform** favors TriGLU. C-Eval and IFEval improve, but a large MGSM deterioration at step 294 creates high TriGLU variance and removes the category-average gain. The source guide's intuition—extra expressivity may improve hard reasoning while occasionally destabilizing simpler knowledge retrieval or multilingual behavior—is consistent with this profile, but is not uniquely established by it.

These local deficits are relative to the strongest source-selected SLRL layer, not to the untuned base model. The broader experiment is an RL improvement over that starting model; a negative TriGLU–baseline cell therefore does not show that EE destroyed the underlying capability. One plausible transfer-specific hypothesis is that the newly initialized EE path received math RL but no broad-domain pretraining, so it can trail the best SLRL comparator on isolated OOD tasks even while improving over the untuned starting point overall. A same-harness per-benchmark base comparison and the staged EE-only warm start in Appendix A are needed to test that interpretation.

The OOD category averages should therefore be treated as deliberately weak summaries. They assign equal weight to benchmarks with different domains, answer formats, random-choice floors, sample counts, and protocol maturity. Such an average can hide a gain on a strict generative diagnostic—most clearly **GPQA-Diamond-Freeform** here—behind a small loss on an unrelated constituent. The raw benchmark vector, protocol status, and cross-checkpoint stability are more informative than the sign of a single unweighted OOD average.

The **GPQA-Diamond-Freeform** table cell is computed from checkpoint-level strict accuracy. Each of the five checkpoints contributes 126 manually audited responses; uncertain judgments count as incorrect. Averaging those five equal-denominator accuracies is equivalent to 40 correct checkpoint–question runs out of 630 for TriGLU (6.349%) and 29 out of 630 for the baseline (4.603%). A separate union diagnostic asks how many distinct questions were solved by at least one checkpoint: 18 of 126 for TriGLU and 12 of 126 for the baseline. That union statistic is useful for capability coverage but is not the value reported in Table 3.

This caution is also consistent with the source paper's Qwen3-1.7B layer sweep: Layer 10 is selected for strong Math contribution, but it is not uniformly the best layer across OOD categories [Zhang et al., 2026]. One plausible interpretation is that functional specialization remains relatively diffuse at this parameter scale, so a mathematically high-contribution layer can still produce heterogeneous transfer. The stronger layer-role separation suggested by larger-model results is a cross-scale hypothesis, not something established by the present single-seed Qwen3-1.7B experiment.

Training reward and downstream evaluation were not monotonically aligned. At several checkpoints the EE variant reached equal or lower recent reward while obtaining stronger held-out metrics. This is evidence against selecting these models by training reward alone. It does not show that lower reward causes better generalization; reward sampling, shuffled-batch composition, KL reference resets, and schedule changes all vary along the same trajectory.

6.3 Why checkpoint averaging is used

From step 158 onward, both variants occupy an oscillatory plateau: different shuffled batches pull the model toward different capability profiles. In a production workflow, researchers often select a checkpoint during such a plateau. Averaging matched checkpoints therefore reduces dependence on one post-hoc checkpoint choice and describes the typical observed trajectory. However, five checkpoints from one run are not five independent experiments. No claim of statistical significance is made, and a multiple-seed replication is the most important missing validation.

6.4 Full corrected checkpoint table

Table 4 provides the complete per-checkpoint record. The native LaTeX table is generated from the repository's machine-readable consolidated table; it excludes SFT, untuned-base, and early RL checkpoints by design. The

report-specific renderer mutes every equal-weight Avg column and emphasizes **OlympiadBench**, **LiveCodeBench**, and **GPQA-Diamond-Freeform**; this visual hierarchy is presentation metadata, not a change to any score.

The body contains ten matched cells: five checkpoints for TriGLU and five for the baseline. The final rows report the population mean and standard deviation across checkpoints within each trajectory. These are descriptive trajectory summaries rather than multi-seed confidence estimates.

All displayed values use the corrected protocol where a correction was available. HumanEval+ uses the raw no-chat route, and LiveCodeBench uses the repaired execution contract. MBPP and its dependent CodeAvg remain marked provisional because their full protocol matrix is incomplete. MathAvg is computed from MATH-500, GSM8K, OlympiadBench, and AMC Average@32; the adjacent AMC greedy column is intentionally outside that aggregate. Likewise, GPQA-Diamond-Freeform is shown beside ReasoningAvg but is excluded from it. The table retains response-cap-inclusive values, so it should be read together with Section 5.4 rather than as an uncapped capability ceiling.

Table 4: Full corrected Math and OOD results at matched RL steps 158, 196, 226, 256, and 294, including across-checkpoint means and population standard deviations. MathAvg uses AMC Average@32, not AMC greedy. ReasoningAvg excludes GPQA-Diamond-Freeform. All Avg columns are intentionally muted; the three emphasized columns are hard generative benchmarks. MBPP and CodeAvg remain provisional.

Setting	Math						Code				Reasoning				Language			
Setting	MATH-500	GSM8K	**Olympiad Bench**	AMC Avg@32	MathAvg†	AMC greedy	HumanEval+ corrected	MBPP*	**LiveCodeBench corrected**	CodeAvg*†	GPQA-Diamond	MMLU-Pro	ReasAvg†	**GPQA-Diamond-Freeform**	C-Eval	IFEval	MGSM	LangAvg†
TriGLU-158	69.200	82.942	**26.815**	33.359	53.079	40.000	59.146	30.401	**10.995**	33.514	26.766	33.991	30.378	**5.556**	46.582	28.513	59.417	44.837
TriGLU-196	66.800	82.714	**28.296**	33.672	52.871	37.500	61.585	28.601	**10.807**	33.664	26.768	34.259	30.513	**4.762**	47.399	26.834	57.564	43.932
TriGLU-226	66.200	82.638	**29.778**	33.828	53.111	37.500	60.976	30.599	**10.903**	34.159	25.254	34.101	29.677	**5.556**	47.548	26.624	60.146	44.773
TriGLU-256	68.800	83.548	**26.370**	32.109	52.707	37.500	61.585	27.600	**11.376**	33.521	24.242	34.353	29.297	**7.937**	48.218	27.673	59.566	45.152
TriGLU-294	66.600	82.866	**29.037**	34.375	53.219	37.500	60.976	29.200	**10.142**	33.439	29.800	32.770	31.285	**7.937**	47.990	27.460	46.000	40.483
TriGLU mean ± std	67.52 ± 1.23	82.94 ± 0.32	**28.06 ± 1.29**	33.47 ± 0.76	53.00 ± 0.18	38.00 ± 1.00	60.85 ± 0.90	29.28 ± 1.12	**10.84 ± 0.40**	33.66 ± 0.26	26.57 ± 1.88	33.89 ± 0.58	30.23 ± 0.69	**6.35 ± 1.33**	47.55 ± 0.57	27.42 ± 0.67	56.54 ± 5.34	43.84 ± 1.72
Baseline-158	66.800	82.942	**26.370**	33.750	52.465	35.000	59.756	29.598	**9.764**	33.039	28.283	34.432	31.358	**6.349**	45.392	26.416	58.909	43.572
Baseline-196	67.200	82.032	**24.889**	32.656	51.694	37.500	60.976	28.202	**9.764**	32.980	22.727	34.533	28.630	**4.762**	45.692	27.046	59.017	43.918
Baseline-226	67.400	82.638	**25.778**	35.156	52.743	32.500	60.366	29.600	**10.046**	33.337	25.250	34.027	29.638	**3.175**	46.284	25.998	58.726	43.669
Baseline-256	66.800	81.729	**25.037**	33.828	51.848	40.000	57.927	28.800	**9.667**	32.131	27.778	34.466	31.122	**3.968**	46.731	27.884	58.838	44.484
Baseline-294	66.200	82.638	**25.481**	35.156	52.369	32.500	58.537	27.401	**9.670**	31.869	28.788	34.143	31.465	**4.762**	46.808	26.417	59.309	44.178
Baseline mean ± std	66.88 ± 0.41	82.40 ± 0.45	**25.51 ± 0.53**	34.11 ± 0.95	52.22 ± 0.39	35.50 ± 2.92	59.51 ± 1.13	28.72 ± 0.84	**9.78 ± 0.14**	32.67 ± 0.57	26.57 ± 2.27	34.32 ± 0.20	30.44 ± 1.12	**4.60 ± 1.05**	46.18 ± 0.56	26.75 ± 0.66	58.96 ± 0.20	43.96 ± 0.33

Evidence hierarchy. Bold columns are the three hard generative benchmarks: OlympiadBench, corrected LiveCodeBench, and GPQA-Diamond-Freeform. All Avg columns are muted because they are naive equal-weight descriptive summaries, not model-selection objectives.

GPQA-Freeform accounting. Each checkpoint is scored over 126 manually audited questions. The displayed architecture mean is equivalent to 40/630 correct checkpoint–question runs for TriGLU and 29/630 for baseline; the separate distinct-question union is 18/126 versus 12/126 and is not the table score.

Protocol boundary. HumanEval+ and LiveCodeBench use corrected routes. MBPP and therefore CodeAvg remain provisional. Cap-hit-inclusive results are retained for matched comparison.

7 Systems Integration and Efficiency

The study required an architecture-aware serving route rather than only a training-time PyTorch patch. The repository provides an explicit `Qwen3TriGLUForCausalLM`, an out-of-tree vLLM plugin, export metadata, dispatch receipts, post-load FP32 restoration for the side branch, live weight synchronization, and exact greedy-token parity in bounded HF/vLLM gates. The production custom math still executes as ordinary PyTorch/cuBLAS operations inside vLLM; no production TriGLU Triton kernel is claimed.

On two RTX 5090 GPUs, using independent TP=1 vLLM replicas, pressure-64 matched long decoding reached 2,872.1 generated tokens/s/GPU for TriGLU versus 5,442.1 for vanilla Qwen, or 52.8% of the vanilla rate. The same TriGLU path was 151.8% of the historical SHS reference path (1,892.1 tokens/s/GPU). Thus the architecture is substantially slower than the baseline even though it modifies only one layer: autoregressive decoding repeatedly invokes that layer, and the FP32 side path adds six projections, three nonlinear streams, elementwise products, and conversion/kernel-launch overhead at every generated token.

This result motivates, but does not yet validate, three optimizations: a pure BF16 architecture path, fused side-branch kernels, and a native registered serving implementation that minimizes framework transitions. Continuous batching is already demonstrated and is architecture-agnostic at the scheduler level; what must be integrated is the model's custom forward path, weight loading, export metadata, and synchronization semantics.

If backend sensitivity remains after strict parity work, a second deployment path is to use the high-capacity EE model as a teacher rather than the final serving artifact. Calibrated logit or sequence distillation into a conventional student could preserve part of the learned capability while recovering mature kernels and serving support. This is a fallback research direction, not an explanation of the present gains, and it requires its own temperature, calibration, and teacher–student fidelity study.

8 Limitations

1. **One seed.** Checkpoint averaging cannot substitute for independent training seeds.
2. **Live schedule changes.** Learning-rate decay and KL-reference resets were introduced during the run. They are matched between variants, but prevent a clean claim about the original paper schedule.
3. **Protocol divergence.** The study did not have an executable paper repository. Local absolute scores differ from the paper, and several OOD evaluators required protocol repairs.
4. **Cap hits.** Training and evaluation contain truncated generations. High-cap OOD cells are not capability ceilings.
5. **Provisional MBPP.** CodeAvg inherits MBPP's unresolved protocol boundary.
6. **Manual free-form audit.** GPQA-Diamond-Freeform provides a strict complementary diagnostic, but its semantic judgments include 13 uncertain responses counted as incorrect and are not directly interchangeable with MCQ accuracy.
7. **Capacity mismatch.** TriGLU adds 41.69% of one layer's parameters. The experiment tests this concrete architecture, not a parameter-matched universal EE principle.
8. **Precision and backend sensitivity.** The FP32 side branch and generic PyTorch/cuBLAS execution affect both throughput and numerical behavior. Pure-BF16 and strict HF/vLLM parity remain pending.

9 Future Work

The immediate items below arise directly from the completed study. A broader post-study research agenda, developed after these experiments and therefore not part of their evidence, is summarized separately in Appendix A. That appendix preserves the distinction between measured findings, retrospective interpretation, and unvalidated architectural proposals.

9.1 Immediate controlled experiments

The next scientific priorities are:

- repeat the matched baseline/TriGLU comparison across multiple seeds;

- complete the HF/vLLM evaluation parity matrix and cap-repair program;
- train pure-BF16 TriGLU and SHS variants, with accuracy and throughput gates;
- compare lower EE levels by reducing side width or the number of multiplicative streams;
- run OFT only under a matched backbone-adaptation policy, separating geometry preservation from trainable-capacity differences;
- replace fixed GeLU components with trainable polynomial activations, informed by PolyCom/PolyNorm [Zhuo et al., 2024];
- test per-token HyperGrid- or HyperNetwork-style modulation, without sequence pooling and beginning with factorized forms rather than full dynamic weight generation.

9.2 Joint adaptation and expressivity matching

The current intervention assumes that frozen surrounding layers can consume a representation produced by a more expressive Layer 10. The MGSM instability suggests that this interface can become a bottleneck. Two extensions follow. First, train adjacent layers jointly, perhaps with full updates near Layer 10 and constrained updates farther away. Second, add progressively weaker EE modules to neighboring layers, analogous to impedance matching: rather than a single abrupt change in representational geometry, increase and decrease the available interaction order gradually across depth.

The layer-selective paper reports that guided multi-layer updates are especially strong on larger Qwen3 models, including a best-10 strategy on Qwen3-8B [Zhang et al., 2026]. A direct Qwen3-8B B10 EE-PEFT comparison is therefore a high-value follow-up. It would provide more layers in which the added architecture can co-adapt and test whether the present single-layer gains are limited by surrounding-layer geometry.

9.3 Fresh pretraining

The cleanest test of EE is fresh pretraining, where every layer can adapt to the new computation. It removes the need for a residual $(1 + \delta)$ constraint and permits side streams with scale comparable to the main stream. A compute-aware program would begin with a dense model near one billion parameters, compare SwiGLU, reduced-branch TriGLU, polynomial activations, and factorized HyperGrid variants under matched FLOPs, and only then scale to MoE or recurrent architectures.

9.4 Scaling perspective

The long-term motivation is not that EE removes attention cost. Full attention and KV-cache requirements remain major constraints, and sparse or linear attention may only replace part of a production model's dense attention. The claim is narrower: for a fixed attention burden, richer FFN interactions can increase the model's representational options with compute and activation memory that scale linearly in sequence length. SwiGLU's success over simpler MLPs is one precedent; whether more aggressive EE mechanisms preserve a useful quality/throughput frontier at large scale remains an empirical question.

10 Conclusion

We proposed *Expressivity Engineering* from a multiplication-first distinction: fixed linear maps mix channels, while self-multiplication, multiplication by a transformed self, and state-generated weights create higher-order or input-conditioned interaction. This lens places nonlinear activations, GLUs, attention, HyperNetworks, MoE, and MoLoRA along one shared design axis without claiming that the modules are interchangeable. Using the layer-localization and experimental workflow of Zhang et al. [2026] as guide rails, we tested one high-capacity EE module at Qwen3-1.7B-Base Layer 10 under matched single-layer GRPO. The intervention starts as an exact functional no-op and differs from the baseline only through the added side architecture and its parameters.

Across five matched checkpoints, the EE variant improves the primary Math average and the provisional Code average, with especially consistent gains on **OlympiadBench**, **corrected LiveCodeBench**, and strict **GPQA-Diamond-Freeform**. It does not uniformly dominate: aggregate Reasoning and Language are tied or slightly lower, MGSM contains a severe late outlier, and serving throughput is roughly half that of vanilla Qwen under the

measured setup. The appropriate conclusion is therefore a positive, artifact-backed pilot: added FFN expressivity can improve a strong single-layer RL baseline on several hard tasks, but the result still requires multi-seed replication, protocol closure, precision ablation, and efficiency engineering before it can support a broad claim.

A Follow-up Research Agenda

A.1 Provenance and evidence boundary

This appendix condenses a research agenda formulated after the completed Qwen3-1.7B study, including retrospective connections made after reading the Kimi K3 and LatentMoE reports [Kimi Team, 2026, Elango et al., 2026]. It does not retroactively change the design motivation, protocol, or claims of the main study. None of the proposals below has been validated by the reported runs. Their common premise is the multiplication-first definition of *Expressivity Engineering*: fixed linear maps mix channels, whereas input-conditioned multiplication, dynamic operators, and compositions of nonlinear components introduce higher-order or context-dependent interactions. The complete provenance and long-form derivations remain in the repository follow-up note; this appendix retains the mechanisms, corrections, caveats, and falsification requirements needed to evaluate them.

A.2 Bottlenecks and structured dynamic parameterization

The implemented TriGLU side path maps the 2,048-dimensional Layer-10 state through a 512-dimensional bottleneck, expands it into three 2,048-dimensional streams, multiplies those streams element-wise, and returns a 6,144-dimensional bounded modulation. This bottleneck was selected from earlier toy-model ablations, before LatentMoE. Its later resemblance to LatentMoE is substantive but limited: LatentMoE compresses routed expert computation to reduce parameter traffic and all-to-all communication, whereas TriGLU compresses an always-active side modulator while retaining the full-dimensional base SwiGLU path [Elango et al., 2026].

Writing the side return as $s(x) = f(Ax)$ with $A \in \mathbb{R}^{512\times 2048}$ gives, at differentiable points,

$$J_s(x) = J_f(Ax)A, \qquad \operatorname{rank} J_s(x) \le 512. \tag{13}$$

The new degree of freedom must therefore coordinate its output through a shared low-dimensional representation. This may impose structured-feature pressure, suppress nuisance directions, improve conditioning, or reinvest saved input capacity into wider nonlinear streams. "Low-pass filter" is only an analogy: without an ordered frequency basis, the defensible statement is rank-constrained learned compression. A causal test must compare no bottleneck, widths from 256 to 2,048, parameter/FLOP-matched stream widths, and a low-rank linear control, while reporting quality, throughput, Jacobian spectra, activation covariance rank, and noise sensitivity.

A lower-parameter SHS successor can replace HyperGrid modulation with a hierarchy of structured HyperNetwork gates. For output-channel gating,

$$y = [1 + g(x)] \odot (Wx), \tag{14}$$

where a zero-initialized generator head gives an exact no-op. A richer rank-R entry-wise gate is

$$G(x) = \sum_{r=1}^{R} a_r(x) b_r(x)^\top, \qquad W_{\text{eff}}(x) = W \odot [1 + G(x)], \tag{15}$$

and an additive multi-LoRA HyperNetwork can use $\Delta W(x) = \sum_\ell \alpha_\ell(x) A_\ell B_\ell$. The required comparison is channel-wise diagonal, rank-one, rank-R, dynamic mixtures of fixed LoRA bases, and shuffled HyperGrid under matched parameters and FLOPs. Parameter savings come from diagonal structure, low-rank factorization, or basis sharing, not from generating a literal dense $d_{\text{out}} \times d_{\text{in}}$ matrix.

A.3 Expert composition and dynamic SwiGLU components

Sparse MoE conventionally forms a router-weighted sum. A distinct hypothesis is to combine bounded near-identity expert factors,

$$m(x) = \prod_{j\in M(x)} [1 + \epsilon_j(x) \tanh E_j(x)], \quad y = x + \sum_{i\in A(x)} p_i(x) E_i(x) + \beta(x) \odot [m(x) - 1]. \tag{16}$$

Shared and routed experts could be assigned additive or multiplicative roles, or several shared experts could multiply before modulating the routed aggregate. Bounded branches, normalization, and exact-no-op or near-identity initialization are essential because raw products can produce exploding or vanishing activations and gradients.

CartesianMoE sequentially composes two routed sub-MoE layers, while Product of Experts multiplies normalized probabilistic factors; both are close precedents but neither is the same as an element-wise product of hidden expert representations in one aggregation stage [Su et al., 2025, Hinton, 2002]. Kimi K3's Quantile Balancing weakens load imbalance as an argument against sparse routing, but does not guarantee semantic diversity or equal learning progress [Kimi Team, 2026].

A full-rank dynamic SwiGLU can instead mix matrix triplets in weight space, $W_{p,\text{eff}}(x) = W_{p,0} + \sum_e \alpha_e(x) W_{p,e}$ for $p \in \{\text{gate}, \text{up}, \text{down}\}$, or evaluate separate nonlinear experts and combine their outputs. These are not equivalent: pre-adding static linear matrices collapses to one matrix, whereas separately evaluated SwiGLUs retain distinct nonlinear functions. Dense nonzero coefficients also expose every component to gradient, avoiding discrete-routing starvation, but they do so by surrendering sparse conditional compute and can still learn redundant components.

The compute boundary is important. With $d_{\text{ff}} = 4d$, one SwiGLU costs approximately $12d^2$ multiply-accumulates per token, not $12d^3$. Sparse top-k MoE costs approximately $12nkd^2$ for n tokens; activating all E experts costs approximately $12nEd^2$. If one coefficient set is shared across a sequence or segment, forming an effective full-rank matrix mixture and then reusing it costs approximately $12Ed^2 + 12nd^2$, which can amortize when $E + n < kn$. Per-token coefficients, however, restore the approximately $12nEd^2$ cost and defeat conditional compute. Full-rank aggregation is therefore a complementary dense EE module or context-shared control, not a replacement for sparse MoE.

A.4 Attention alternatives as Expressivity Engineering

Attention itself is an EE mechanism because token-dependent weights modulate and mix values. Kimi Delta Attention is a recurrent fast-weight mechanism with linear sequence scaling and fixed-size state, while Mamba makes the discretization step Δ_t and write/read terms B_t, C_t depend on the current input [Kimi Team, 2025, Schlag et al., 2021, Gu and Dao, 2023]. In schematic form,

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t. \tag{17}$$

Thus Mamba implements input-conditioned transition, write, and read behavior without regenerating every underlying parameter. The correct systems target is no sequence-length-growing KV cache, not zero memory: recurrent state, generation compute, activation bandwidth, and training intermediates remain.

Candidate replacements include HyperNetworks that generate low-rank write and erase updates, rank-R gates over recurrent transitions, bounded multi-branch write products, shared generated bases across layers, and hybrid stacks that replace only selected attention layers. They must be compared with KDA and Mamba at matched training FLOPs and activated parameters, recurrent state and bandwidth, prefill/decode throughput, numerical stability, parallel training efficiency, long-context retrieval, and overwrite behavior. A stateless per-token HyperNetwork has no long-range memory unless it consumes a context summary or state; once state is added, the experiment compares alternative fast-weight memories rather than "memory-free" attention.

A.5 Training-free Loop as an independent inference intervention

Plain Looping is independent of frequency-shaped EE. It reuses an existing layer or contiguous block without requiring any new EE component or depth-varying EE-level schedule. Conversely, frequency-shaped EE can use independent parameters at every depth and need not loop any block. Combining the two is an optional factorial intervention, not part of either definition.

Training-Free Looped Transformers formalizes the plain-Loop case for frozen released checkpoints [Chen et al., 2026]. For a contiguous window $[a, b]$, define

$$g = L_b \circ \cdots \circ L_a, \qquad F_g(x) = g(x) - x. \tag{18}$$

The ordinary checkpoint applies g once between unchanged pre-window and post-window layers. The wrapper adds no parameters and performs no fine-tuning, continued training, SFT, RL, or other weight update. Block mode repeats the whole window, $(L_b \circ \cdots \circ L_a)^K$, whereas layer mode repeats each layer before moving on, $L_b^K \circ \cdots \circ L_a^K$. Dense models behave broadly similarly under the two modes; layer mode is safer for MoE because block-mode perturbations can change router decisions between iterations and accumulate routing noise.

The Euler interpretation begins with the identity

$$g(x) = x + F_g(x). \tag{19}$$

This has exactly the algebraic form of one forward-Euler step of size $h = 1$ for the virtual residual ODE $\dot{x} = F_g(x)$. The ODE is a numerical interpretation of the residual map, not a claim that the Transformer follows a known physical dynamical system. The frozen checkpoint targets the one-step endpoint

$$x_1 = x_0 + F_g(x_0) = g(x_0), \qquad t : 0 \to 1. \tag{20}$$

Naively applying g a total of K times takes K full steps, $x_{k+1} = g(x_k)$, and therefore approximates virtual time $t = K$. The post-window layers were trained to consume the $t = 1$ state, not this extrapolated endpoint. This explains why naive reapplication commonly drifts out of the trained low-loss region.

Damped Euler instead divides the same total interval $[0, 1]$ into K smaller steps:

$$x_{k+1} = x_k + \frac{1}{K} F_g(x_k) = \left(1 - \frac{1}{K}\right) x_k + \frac{1}{K} g(x_k). \tag{21}$$

For $K = 2$, each update lies halfway between the current state and the full frozen-window output. Two half-steps still target $t = 1$; they do not advance to $t = 2$. This is the intuition behind $\alpha = 1/K$, including $\alpha = 0.5$ for $K = 2$. The paper's practical Runge–Kutta construction also interpolates between the checkpoint's original one-step output and the K-substep damped-Euler endpoint using an anchor weight β: $\beta = 1$ preserves the original output, $\beta = 0$ uses the fully substepped endpoint, and intermediate values remain biased toward the trained endpoint. Higher order is not automatically safer; the paper reports failed wide-window and solver configurations.

The Qwen3-1.7B-Base window-size sweep uses $K = 2$ forward-Euler substeps. Its four-layer window $[12, 15]$ improves the reported 16-task aggregate by $+0.55$ points, while six layers give -0.82, twelve give -0.63, and all 28 layers collapse by -27.73. The broader fixed recipe uses the middle four layers and a three-stage Runge–Kutta strategy. These results motivate a narrow middle-window search but do not imply that the best Loop window, the best RL layer, or an EE peak must coincide.

If the selected window accounts for fraction w of baseline forward cost and is evaluated K times total, only $K - 1$ evaluations are extra:

$$C_{\text{loop}}/C_{\text{base}} = 1 + (K - 1)w. \tag{22}$$

For $w = 0.25$ and $K = 2$, the baseline already pays for the first quarter-window pass and the Loop adds one more, giving $1 + 0.25 = 1.25\times$ total cost rather than $2\times$. With $K = 3$, two extra quarter-window passes give $1.50\times$. This first-order estimate excludes unequal layer costs, caches, routing, kernels, and orchestration overhead.

EE can be added only after the plain-Loop axis is visible. In *EE then Loop*, EE is trained without recurrence and the damped/RK wrapper is applied post hoc to the frozen result. In *Loop-mounted EE*, the same Loop is active while EE trains and can co-adapt to repeated hidden states. The minimum factorial comparison is no Loop/no EE, Loop only, EE only, EE then Loop, and Loop-mounted EE, with matched window, K, integrator, data, and activated compute.

A.6 Frequency-shaped EE across independently parameterized depth

HRM and HRM-Text motivate hierarchical recurrence through fast local and slow high-level processing [Wang et al., 2025, 2026]. The verified HRM-Text schedule is $(L, L, L, H) \times 2$ (H2L3), so one forward pass reuses L six times and H twice. Recurrence increases computation without adding independent weights. The approximately 3.6 bits per parameter measured in a controlled memorization study is not a universal storage constant, but it sharpens a testable tradeoff: tied recurrence may improve iterative reasoning per parameter while offering less independent weight capacity than an untied model at equal active compute [Morris et al., 2025]. This motivates the frequency analogy below, but recurrence is not part of the frequency-shaped EE definition.

The follow-up introduces a provisional EE level because the main report did not define one. Ordinary SwiGLU is Level 1; each additional *sequential* input-conditioned multiplicative composition along a path raises the level by one. Parallel modulation sites are tracked separately. In particular, D_{mul} records maximum sequential multiplicative depth and S_{mod} records the number of separately modulated tensors. SHS may have one controller and $S_{\text{mod}} = 3$ across gate/up/down, but whether that behaves like one or three effective levels is an empirical, budget-matched question.

Define a low-amplitude detail block and a broader abstraction block as

$$L_{\mathrm{EE}} = [1, 2, 2, 1], \qquad H_{\mathrm{EE}} = [1, 2, 3, 4, 3, 2, 1]. \tag{23}$$

The proposed depth-domain analogue of H2L3 is $(L_{\mathrm{EE}}, L_{\mathrm{EE}}, L_{\mathrm{EE}}, H_{\mathrm{EE}}) \times 2$. Here "frequency" means structured variation over network depth, not a Fourier frequency or literal brain oscillation. The default realization uses independent parameters at every depth: the two copies contain six independent L_{EE} blocks and two independent H_{EE} blocks. This is an ordinary feed-forward depth schedule, not a Loop. It increases checkpoint, optimizer, communication, memory, and data requirements relative to tied recurrence, but retains more independent weight capacity. A tied version that reuses the same L_{EE} and H_{EE} blocks is a separate Loop-plus-frequency experiment governed by Eq. 21, not the default frequency-shaped EE proposal. Native pretraining is the primary test; cloning middle blocks with exact-no-op EE branches is a separate retrofit experiment.

Attention type supplies a second depth-domain pattern. Kimi K3 contains 69 KDA and 24 Gated MLA layers, approximately a three-KDA-to-one-full cadence [Kimi Team, 2026]. A phase-locked hypothesis assigns lower FFN EE levels to efficient-attention layers and a higher level to periodic Full-Attention layers, for example

$$[\mathrm{KDA}_1, \mathrm{KDA}_1, \mathrm{KDA}_1, \mathrm{Full}_3]. \tag{24}$$

This synchronizes broader token mixing with a higher-order FFN integration event and may reproduce part of an HRM-like detail/integration hierarchy without weight sharing. The minimum controls are attention-only $[1, 1, 1, 1]$, elevated aligned $[2, 2, 2, 4]$, a budget-matched shifted peak $[1, 1, 3, 1]$, uniform-level, and shuffled-peak schedules. The raw diagnostic anti-phase $[3, 3, 3, 1]$ is not budget-matched and must not be treated as the primary control. HydraHead offers a distinct within-layer alternative by mixing Full and Linear Attention across heads; it must not be conflated with the earlier Hydra Attention mechanism [Tan et al., 2026, Bolya et al., 2022]. Holding a trained HydraHead full/linear head ratio constant across depth would isolate the FFN-level envelope; varying the ratio jointly with FFN level would test whether their peaks should align, alternate, or remain independent.

A.7 A CPT–SFT–RL warm start for EE-PEFT

The reported direct-RL run is a cold start for the new EE component. Its zero-initialized return makes the initial model an exact functional no-op, but its internal parameters have received no broad contiguous-document pretraining and no general instruction-following supervision before their first task-conditioned gradients arrive from math rollout traces. The result therefore shows that a randomly initialized, non-pretrained EE path can grok part of the RL objective; it does not show that direct RL is the optimal curriculum.

A staged alternative freezes the backbone and trains only the EE component: (1) broad-domain continued pretraining on a decontaminated mixture of books, web text, code, mathematics, multilingual text, and other long-form documents; (2) EE-only SFT on instruction-following and verified chain-of-thought data; and (3) the matched RL protocol. This proposal is distinct from the historical SFT pilot: that pilot updated a broader selected-layer configuration and did not initialize the production RL runs. The minimum ablation is direct RL, CPT-to-RL, SFT-to-RL, and CPT-to-SFT-to-RL, with matched RL prompts and seeds, explicit token/compute accounting, and a separately named backbone-unfrozen condition.

A.8 Falsification matrix and experimental order

Table 5 compresses the acceptance logic. Improvements must survive matched parameters, activated FLOPs, data, and evaluation; where exact matching is impossible, both resource axes and fitted scaling controls must be reported.

Table 5: Post-study proposals and their minimum falsification controls.

Proposal	Mechanism	Expected benefit	Required control	Main risk
TriGLU bottleneck	Rank-constrained side coordinates	Structure and conditioning	Width and no-bottleneck ablations	Under-capacity
Structured SHS	Diagonal, rank-R, or multi-LoRA dynamic gates	Finer modulation per parameter	HyperGrid at matched budget	Generator overhead
Expert products	Bounded multiplicative expert fusion	Higher-order expert interaction	Additive MoE at matched active cost	Numerical instability
Dynamic SwiGLU	Context-shared full-rank basis mixture	Dense dynamic control	Sparse top-k and static merged matrices	Loss of conditional compute
Attention EE	Dynamic recurrent write/read operators	Linear-scaling context processing	KDA, Mamba, and Full Attention	State and bandwidth cost
Plain middle Loop	Frozen block reuse with damped Euler or RK	Inference-time depth refinement	No-Loop checkpoint and naive Loop	State drift and extra latency
Frequency-shaped EE	Untied depth-varying level envelope; phase locking	Detail/integration hierarchy	Flat, shifted, shuffled, and plain-Loop cells	Capacity/compute confounding
EE warm start	EE-only CPT, SFT, then RL	General-language and instruction priors before reward optimization	Direct RL, CPT-only, and SFT-only starts	Extra-token/compute confounding

The staged order is: (1) compare direct-RL cold start with EE-only CPT, SFT, and CPT-to-SFT warm starts; (2) compare diagonal, rank-one, rank-R, multi-LoRA, and HyperGrid gates; (3) test residualized two-expert products before scaling expert count; (4) separate shared/routed fusion policies and compare dense training with QB-balanced sparse routing; (5) validate the plain frozen-model Loop axis and its damped-Euler/RK strategy without added EE; (6) compare untied frequency-shaped EE with flat and shuffled depth schedules; (7) cross the independent Loop and EE axes using post-hoc EE-then-Loop and Loop-mounted EE; (8) test phase-locked attention/FFN schedules against shifted, uniform, and shuffled controls; and only then (9) test full-rank dynamic SwiGLU bases and attention replacement. Every cell should begin as an exact no-op or controlled near-identity perturbation and report independent and activated parameters, training and inference FLOPs, communication, state memory, throughput, factual-knowledge capacity, reasoning, generalization, and sensitivity to additional recurrent steps.

Artifact Availability

The accompanying repository contains architecture code, configs, deterministic data-order contracts, evaluation scripts, compact runtime receipts, manual audit packages, and the corrected consolidated result table. Model weights, datasets, checkpoints, uncurated generations, private operational logs, and chat transcripts are excluded from the research release. The report's numeric source of truth is the repository record `docs/experiment_records/2026-07-18_qwen3-1p7b-math-ood-corrected-consolidated-master-table.md`.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL https://arxiv.org/abs/2108.07732.

Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Hydra attention: Efficient attention with many heads, 2022. URL https://arxiv.org/abs/2209.07484.

Yuxuan Chen et al. Training-free looped transformers, 2026. URL https://arxiv.org/abs/2605.23872.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Vijay Elango et al. LatentMoE: Toward optimal accuracy per FLOP and parameter in mixture of experts, 2026. URL https://arxiv.org/abs/2601.18089.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023. URL https://arxiv.org/abs/2312.00752.

David Ha, Andrew Dai, and Quoc V. Le. Hypernetworks, 2016. URL https://arxiv.org/abs/1609.09106.

Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems, 2024. URL https://arxiv.org/abs/2402.14008.

Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus), 2016. URL https://arxiv.org/abs/1606.08415.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In *Advances in Neural Information Processing Systems*, 2021. URL https://arxiv.org/abs/2103.03874.

Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14 (8):1771–1800, 2002. doi: 10.1162/089976602760128018.

Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Yao Fu, Maosong Sun, and Zhiyuan Liu. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models, 2023. URL https://arxiv.org/abs/2305.08322.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL https://arxiv.org/abs/2403.07974.

Kimi Team. Kimi linear: An expressive, efficient attention architecture, 2025. URL https://arxiv.org/abs/2510.26692.

Kimi Team. Kimi k3: Open frontier intelligence, 2026. URL https://arxiv.org/abs/2607.24653.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL https://arxiv.org/abs/2309.06180.

Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. Numinamath. Hugging Face dataset repository, 2024. URL https://huggingface.co/AI-MO/NuminaMath-CoT.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2305.01210.

Chuancheng Lv, Lei Li, Shitou Zhang, Gang Chen, Fanchao Qi, Ningyu Zhang, and Hai-Tao Zheng. Hyperlora: Efficient cross-task generalization via constrained low-rank adapters generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 16376–16393, 2024. doi: 10.18653/v1/2024.findings-emnlp.956. URL https://aclanthology.org/2024.findings-emnlp.956/.

Rabeeh Karimi Mahabadi, Sebastian Ruder, Mostafa Dehghani, and James Henderson. Parameter-efficient multi-task fine-tuning for transformers via shared hypernetworks. In *Proceedings of ACL-IJCNLP*, pages 565–576, 2021. doi: 10.18653/v1/2021.acl-long.47. URL https://aclanthology.org/2021.acl-long.47/.

Mathematical Association of America. American mathematics competitions: Amc 10 and amc 12. Official competition information, 2023. URL https://maa.org/student-programs/amc/.

John X. Morris et al. How much do language models memorize?, 2025. URL https://arxiv.org/abs/2505.24832.

Zeju Qiu, Weiyang Liu, Haiwen Feng, Yuxuan Xue, Yao Feng, Zhen Liu, Dan Zhang, Adrian Weller, and Bernhard Schölkopf. Controlling text-to-image diffusion by orthogonal finetuning. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2306.07280.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023. URL https://arxiv.org/abs/2311.12022.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers, 2021. URL https://arxiv.org/abs/2102.11174.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Noam Shazeer. Glu variants improve transformer, 2020. URL https://arxiv.org/abs/2002.05202.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017. URL https://arxiv.org/abs/1701.06538.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025. doi: 10.1145/3689031.3696075.

Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. Language models are multilingual chain-of-thought reasoners, 2022. URL https://arxiv.org/abs/2210.03057.

Jianlin Su et al. CartesianMoE: Boosting knowledge sharing among experts via cartesian product routing in mixture-of-experts. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2025. URL https://aclanthology.org/2025.naacl-long.505/.

Zhixing Tan et al. HydraHead: From head-level functional heterogeneity to specialized attention hybridization, 2026. URL https://arxiv.org/abs/2606.20097.

Yi Tay, Zhe Zhao, Dara Bahri, Donald Metzler, and Da-Cheng Juan. Hypergrid: Efficient multi-task transformers with grid-wise decomposable hyper projections, 2020. URL https://arxiv.org/abs/2007.05891.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. URL https://arxiv.org/abs/1706.03762.

Guan Wang et al. Hierarchical reasoning model, 2025. URL https://arxiv.org/abs/2506.21734.

Guan Wang et al. HRM-Text: Efficient pretraining beyond scaling, 2026. URL https://arxiv.org/abs/2605.20613.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024. URL https://arxiv.org/abs/2406.01574.

An Yang et al. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Ted Zadouri, Ahmet Üstün, Arash Ahmadian, Beyza Ermis, Acyr Locatelli, and Sara Hooker. Pushing mixture of experts to the limit: Extremely parameter efficient moe for instruction tuning. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/6d00071564ec447466fc4577743cf1b3-Abstract-Conference.html.

Zijian Zhang, Rizhen Hu, Athanasios Glentis, Dawei Li, Chung-Yiu Yau, Hongzhou Lin, and Mingyi Hong. Is one layer enough? training a single transformer layer can match full-parameter rl training, 2026. URL https://arxiv.org/abs/2607.01232.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023. URL https://arxiv.org/abs/2311.07911.

Zhijian Zhuo, Ya Wang, Yutao Zeng, Xiaoqing Li, Xun Zhou, and Jinwen Ma. Polynomial composition activations: Unleashing the dynamics of large language models, 2024. URL https://arxiv.org/abs/2411.03884.